

Tarea 5 · Pruebas automatizadas

Universidad del Valle de Guatemala

Facultad de Ingeniería · Departamento de Ciencias de la Computación
CC3091 — Ingeniería de Software 2
Semestre II · 2026

Proyecto: ServiGT

Estrategia de integración y regresión con PHPUnit, PostgreSQL y GitHub Actions.

Integrantes y carnés: completar con los datos del equipo.

Fecha de elaboración: 6 de octubre de 2026.

Estado del trabajo

Se implementaron y ejecutaron seis pruebas sobre PostgreSQL: las seis aprobaron en la línea base; la mutación de expiración produjo exactamente el fallo R1; al restaurar el código aprobaron las seis nuevamente. También aprobaron veinte pruebas existentes de los módulos relacionados. Se conservan logs, XML JUnit y HTML de la ejecución local.

El código y los entregables se publicaron en el repositorio tarea5-software. GitHub Actions ejecutó y aprobó la suite automática y la demostración completa de éxito, regresión y corrección. El video sin voz incorpora la evidencia real del proceso remoto. Antes de entregar, completar nombres y carnés del equipo.

1. Contexto y estrategia

ServiGT es una plataforma de servicios con usuarios cliente, proveedor y administrador. El backend utiliza Laravel y Sanctum con PostgreSQL; el frontend utiliza Expo y React Native Web. La copia inspeccionada declara Laravel ^13.17, PHP ^8.3 y PHPUnit ^12.5.12 en composer.json. El esquema efectivo se define en ServiGT/database/init.sql.

Se tomó como referencia el repositorio vecino G6Software-PServicios, commit 7ea97d48f621e3e350e10125530132cba35a2b7d, rama feat/sprint8-owner-d. El repositorio original contiene pruebas Feature y workflows de backend y frontend, que se consideraron como antecedentes. Se incorporaron únicamente archivos nuevos de Tarea 5, preservando los cambios y documentos existentes. Se actualizó origin/dev antes de integrar; no se cambió de rama ni se mezclaron modificaciones ajenas.

La estrategia propuesta añade seis pruebas, separadas por su propósito: tres verifican integraciones y tres protegen comportamientos existentes. Una prueba de regresión puede también cruzar componentes reales; la diferencia es el comportamiento histórico que protege. Se eligieron pedidos, compra simulada de créditos y lectura de notificaciones, presentes en el código y en pruebas anteriores del proyecto.

Qué se ejecuta realmente

Las solicitudes postJson, putJson y getJson recorren el kernel HTTP real de Laravel, sus rutas, controladores, modelos, recursos JSON y conexión PostgreSQL. No se sustituyen los controladores, repositorios ni la base por mocks. Sanctum::actingAs establece el actor autenticado para el test; no comprueba emisión de tokens ni transporte de red. La integración es aplicación/persistencia, no frontend/API.

Cada test usa DatabaseTransactions para revertir sus fixtures. La base se crea desde el SQL real, en un servicio exclusivo con tmpfs, sin puertos publicados. No se usa RefreshDatabase porque las migraciones del backend no representan todo el esquema del proyecto.

Criterios de aceptación

La línea base debe ejecutar seis pruebas sin fallos, errores ni omisiones. La mutación debe producir exactamente un fallo en R1, con una assertion que identifica la duración incorrecta. Después de restaurar el controlador, las seis deben aprobar. Un fallo de infraestructura no se acepta como regresión detectada.

2. Comparación de herramientas de pruebas

Se compararon tres alternativas aplicables al stack real. Las características siguientes se contrastaron con documentación oficial [1–5]. La valoración de adecuación al proyecto es una decisión técnica del equipo, basada en el código inspeccionado.

Criterio	PHPUnit + Laravel	Pest + Laravel	Playwright
Compatibilidad	PHP; ya declarado en require-dev. Integra el kernel y Eloquent.	PHP; puede aprovechar tests Laravel, con dependencias y plugin adicionales.	Node; apropiado para Expo Web en navegador y solicitudes HTTP.
Configuración	composer.lock existente, XML dedicado, fixtures y PostgreSQL.	Instalar Pest/plugin, revisar versiones compatibles y adaptar convenciones.	Instalar paquete y navegadores; levantar frontend/API y preparar datos.
Tipos de pruebas	Unitarias e integración; permite regresión mediante assertions estables.	Unitarias e integración con sintaxis funcional; regresión con los mismos oráculos.	E2E de web e integración HTTP; regresión de flujos visibles. No cubre UI nativa móvil.
Flujo de desarrollo	Se añade a la suite y corre local/CI dentro de Docker.	Puede coexistir con pruebas PHPUnit; adopción exige acordar estilo.	Suite complementaria que depende del stack disponible y selectores estables.
Resultados y reportes	JUnit XML, TestDox HTML y códigos de salida.	Salida del runner; opciones de reporte dependen de la versión y PHPUnit subyacente.	HTML, JUnit y trazas para investigar interacción y red.
Documentación	Manual oficial de PHPUnit y documentación de testing de Laravel.	Guías oficiales de instalación y plugins Laravel.	Documentación oficial de runner, CI y Trace Viewer.
Ventajas	Reutiliza infraestructura; fixtures transaccionales; assertions directas sobre DB.	Sintaxis breve y organización expresiva; facilita adopción por desarrolladores PHP.	Prueba la experiencia web completa; trazas ayudan a explicar fallos.
Limitaciones	No comprueba renderizado ni navegación del frontend; fixtures requieren mantenimiento.	No elimina la preparación de la DB; añadir un framework no amplía por sí solo la cobertura.	Mayor costo de levantar servicios/navegadores; sincronización y selectores pueden volver frágiles los tests.

Selección: PHPUnit. La automatización existente y las funciones elegidas se concentran en backend/persistencia. PHPUnit permite reutilizar las dependencias bloqueadas y verificar respuestas y efectos en varias tablas sin incorporar otro runner. Pest ofrece una mejora de estilo, pero no una cobertura adicional necesaria para estos seis casos. Playwright se mantendría como siguiente etapa para flujos de Expo Web.

3. Comparación de integración continua

Se compararon GitHub Actions, GitLab CI/CD y Jenkins usando documentación oficial [6–10]. Las tres pueden ejecutar pruebas PHP dentro de contenedores; la selección depende del repositorio, la administración disponible y la evidencia requerida.

Criterio	GitHub Actions	GitLab CI/CD	Jenkins
Compatibilidad	Adecuado al repositorio GitHub y workflows ya existentes; Docker en runner Linux.	Compatible mediante runners con Docker y servicios; requiere repositorio/mirror y acceso en GitLab.	Compatible con agentes que tengan Docker/PHP y plugins necesarios.
Configuración	YAML en .github/workflows, eventos, pasos y artifacts.	Archivo .gitlab-ci.yml, jobs, runner, services y artifacts:reports:junit.	Jenkinsfile, servidor/controlador, agentes, credenciales y plugins.
Pruebas soportadas	Cualquier comando cuyo exit code determine el job; integración y regresión.	Igual; servicios facilitan bases temporales para integración.	Igual; pipelines pueden orquestar servicios y publicar resultados.
Flujo de desarrollo	Push/PR, estados de checks y ejecución manual en GitHub.	Pipelines y merge requests en GitLab; webhooks/mirror si se conserva GitHub.	Integración por SCM/webhooks; requiere configurar conexión y permisos.
Resultados	Logs, resumen de job y artifacts. JUnit se conserva como archivo.	Reporte JUnit visible en la interfaz y artifacts descargables.	Plugin JUnit con resultados/tendencias; logs y archivos archivados.
Documentación	Documentación oficial de eventos, workflow runs y artifacts.	Guías oficiales de YAML, runners y reportes.	Guías de Pipeline, Docker y plugin JUnit.
Ventajas	Menor incorporación al flujo actual; evidencia junto al commit y PR.	Visualización nativa de JUnit y configuración de servicios.	Control amplio del entorno y personalización del proceso.
Limitaciones	Cuotas/retención dependen de cuenta y políticas; artifacts no equivalen a dashboard JUnit.	Añadir otra plataforma al flujo GitHub aumenta coordinación y configuración.	El equipo administra actualizaciones, disponibilidad y seguridad del servidor/agentes.

Selección: GitHub Actions. El proyecto ya usa GitHub y tiene CI; se añade un workflow enfocado, sin reemplazar los checks existentes. El costo de configuración es menor que mantener otra plataforma. Docker reproduce backend y PostgreSQL, y upload-artifact conserva evidencia incluso cuando las pruebas fallan. No se fundamentó la decisión en la demostración de clase.

4. Diseño de las pruebas de integración

Las tres pruebas I1–I3 se implementan en Tarea5Test.php. Los datos son sintéticos y se crean dentro de una transacción revertida al terminar cada prueba. Las assertions consultan PostgreSQL o vuelven a leer el modelo persistido.

I1 · Creación de pedido

Componentes: rutas API, StorePedidoRequest, PedidoController, modelos User/Categoria/Pedido, PostgreSQL y PedidoResource. **Condición:** un cliente autenticado puede publicar un pedido asociado al cliente y categoría correctos.

Datos: cliente de factory, categoría de factory, descripción de reparación eléctrica, dirección Zona 10 y urgencia alta. **Acción:** POST /api/pedidos. **Esperado:** HTTP 201, success=true, estado abierto, IDs correctos en JSON y fila con esos mismos IDs y descripción en pedidos. Detecta errores de asignación, persistencia y serialización.

I2 · Compra simulada de créditos

Componentes: Sanctum, CreditoController, PaqueteCredito, CompraCredito, CreditoProveedor, TransaccionCredito y PostgreSQL. **Condición:** la compra registra sus efectos coherentemente en tres tablas.

Datos: proveedor con saldo 2; paquete sintético con precio Q115, 25 créditos base y 5 bonus; clave tarea5-integracion. **Acción:** POST /api/creditos/comprar. **Esperado:** HTTP 201, compra completada, 30 créditos otorgados y saldo 32. Se comprueba compra del proveedor/paquete, saldo actualizado y transacción tipo compra por 30, referenciada al ID de compra. Este caso verifica el camino exitoso; no demuestra rollback ni concurrencia.

I3 · Lectura y contador de notificaciones

Componentes: NotificacionController, modelo Notificacion, PostgreSQL y JSON de listado. **Condición:** leer una notificación actualiza persistencia y el contador que consume la interfaz.

Datos: cliente y una notificación no leída nueva_solicitud. **Acciones:** GET /api/notificaciones; PUT /api/notificaciones/{id}/leer; GET de nuevo. **Esperado:** HTTP 200 en las tres acciones, contador inicial 1, leida=true al refrescar el modelo y contador final 0. Detecta fallos en actualización y consultas posteriores.

5. Diseño de las pruebas de regresión

Los antecedentes se identificaron en PedidoCreationTest, CreditoComprarTest y NotificacionLecturaTest del repositorio inspeccionado. Esas tres clases se ejecutaron sobre PostgreSQL: veinte pruebas y sesenta y siete assertions aprobadas, además de las seis pruebas nuevas.

Caso	Comportamiento protegido	Cambio que puede romperlo	Oráculo implementado
R1 · Expiración	El pedido expira exactamente 7 días después de crearse.	Cambiar addDays(7) a addDays(1), o alterar serialización/fecha.	Reloj congelado al segundo; comparación de fecha persistida y fecha ISO de respuesta con now()+7 días.
R2 · Idempotencia	Repetir una compra del mismo proveedor y clave no duplica créditos ni historial.	Eliminar la consulta de compra existente o volver a acreditar en el reintento.	Primera solicitud 201; segunda 200; mismo ID; saldo 32; una compra y un movimiento.
R3 · Aislamiento	Un usuario ajeno no puede marcar la notificación de otro ni obtenerla en su listado.	Eliminar el filtro de destinatario del controlador.	PUT responde 404; notificación sigue sin leer; listado ajeno vacío y contador 0.

Experimento deliberado R1

El script ejecuta primero las seis pruebas. Solo si esa línea base aprueba, sustituye una única ocurrencia del código de producción en PedidoController:

```
'fecha_expiracion' => now()->addDays(7),  
// cambio temporal deliberado:  
'fecha_expiracion' => now()->addDays(1),
```

La siguiente ejecución debe fallar exclusivamente en test_R1_pedido_conserva_expiracion_exacta_de_siete_dias. Se verifica el código de salida 1, el total de seis casos, un solo fallo, ningún error/omisión y el mensaje de la assertion sobre siete días. Luego se restauran los bytes originales y se vuelven a ejecutar las seis pruebas.

La restauración también se intenta en finally si surge un problema normal del script. Una interrupción forzada del proceso o pérdida de energía exige revisar el archivo antes de continuar. La mutación pertenece solo a la demostración en una copia de trabajo; no es un cambio de regla de negocio permanente.

6. Automatización implementada

compose.tarea5.yml define únicamente db_tarea5 y pruebas. El servicio PostgreSQL 16 aplica init.sql y usa tmpfs. El backend se construye con el Dockerfile y composer.lock del proyecto. El healthcheck debe aprobar antes de ejecutar PHPUnit. No se necesita levantar el frontend ni la base de desarrollo.

La configuración XML dedicada fuerza pgsql, host db_tarea5 y base tarea5_test. El phpunit.xml original fuerza SQLite en memoria, mientras varios comentarios de CI sostienen que el entorno de Compose lo evita. Para no depender de esa contradicción se invoca explícitamente el XML nuevo. No se modifica el XML original ni se usa una base real.

```
cd tarea5/proyecto
python3 scripts/tarea5.py --mode suite
python3 scripts/tarea5.py --mode demo
```

El modo suite ejecuta seis casos y termina en 0 solo si todos aprueban. El modo demo exige la secuencia 6 aprobadas → R1 fallida → 6 aprobadas. El modo mutant introduce la misma regresión y devuelve 1 tras detectarla, para obtener una ejecución roja de CI. El controlador siempre se restaura al finalizar normalmente.

Eventos y artefactos

El workflow `.github/workflows/tarea5.yml` ejecuta la suite automáticamente ante push o pull request a main o dev. `workflow_dispatch` permite elegir suite, demo o mutant. Un error del runner o de las pruebas hace fallar el job. El modo demo termina verde solo cuando valida las tres fases; el modo mutant termina rojo deliberadamente.

Por cada etapa se generan logs, JUnit XML y TestDox HTML. `resumen.json` registra fecha UTC, duración por etapa, número de casos/fallos/errores, código de salida, commit de CI y hash del controlador. Actions sube los archivos con `if: always()` y retención configurada de 30 días. Las políticas de la cuenta pueden limitarla. Se debe descargar la evidencia antes de su vencimiento.

La limpieza usa un archivo Compose independiente y un proyecto exclusivo; solo elimina sus servicios de pruebas. No ejecuta `down -v` sobre el stack de ServiGT. No correr simultáneamente este script en la misma copia, porque usa el mismo proyecto de Docker y modifica temporalmente un controlador.

Para mostrar tres jobs de CI: correr suite, mutant y suite sobre el mismo commit, conservando cada run y sus artifacts. La ejecución manual requiere que el workflow esté en la rama predeterminada y permisos de escritura [11].

7. Evidencia real de ejecución

Las ejecuciones siguientes se realizaron localmente con Docker, PHP 8.3.32, PHPUnit 12.5.34 y PostgreSQL 16. El comando fue `python3 scripts/tarea5.py --mode demo`. El script terminó en 0 al verificar las tres fases y confirmó que el controlador quedó restaurado. Las fechas de los archivos se registran en UTC; 01:55 UTC del 7 de octubre corresponde a 19:55 del 6 de octubre en Guatemala.

Etapa	Resultado observado	Duración del comando
01-exitosa	6 pruebas; 0 fallos; 0 errores; exit 0	3.409 s
02-regresion	6 pruebas; 1 fallos; 0 errores; exit 1	1.600 s
03-corregida	6 pruebas; 0 fallos; 0 errores; exit 0	1.603 s

01-exitosa: seis pruebas y treinta y tres assertions aprobadas. 02-regresion: seis ejecutadas, treinta y dos assertions, un único fallo en R1. 03-corregida: seis pruebas y treinta y tres assertions aprobadas. La fase fallida produjo la assertion: El pedido debe expirar exactamente siete dias despues de su creacion.

Línea base: OK (6 tests, 33 assertions)
Mutación: Tests: 6, Assertions: 32, Failures: 1.
Después de fijar: OK (6 tests, 33 assertions)

Los archivos 01-exitosa, 02-regresion y 03-corregida se adjuntan como .log, .xml y .html en evidencia-real. resumen.json permite comprobar resultados y tiempos. La suma de comandos de las tres fases es aproximadamente 6.612 segundos, incluida la espera inicial de PostgreSQL; excluye build y limpieza. El tiempo interno de PHPUnit fue 0.434, 0.416 y 0.404 segundos, respectivamente.

Verificación adicional

Se ejecutaron PedidoCreationTest, CreditoComprarTest y NotificacionLecturaTest: veinte pruebas, sesenta y siete assertions, todas aprobadas; PHPUnit tomó 0.887 segundos. El comando completo tomó 3.847 segundos, incluyendo la preparación de la DB. Se adjunta 04-suite-relacionada.log/xml. `php -l` aprobó el archivo de pruebas; Docker Compose, Python, XML y YAML también pasaron la revisión estática.

En el primer intento Docker fue bloqueado por permisos. Tras ampliar el acceso se pudo ejecutar. La primera ejecución mostró una advertencia PSR-4 por el nombre de carpeta y otra por caché en un montaje de solo lectura: se corrigieron la capitalización Tarea5 y cacheResult=false. Se repitieron las tres fases sobre esa versión final, ya sin esas advertencias.

Evidencia remota verificada

[Suite automática por push a main](#) · [Demostración remota en GitHub Actions](#).

Etapa de CI	Resultado	Duración del comando
01-exitosa	6 pruebas; 0 fallos; exit 0	8.062 s
02-regresion	6 pruebas; 1 fallo; exit 1	1.244 s
03-corregida	6 pruebas; 0 fallos; exit 0	1.226 s

GitHub Actions aprobó la suite disparada automáticamente por el push a main y la demostración manual sobre el mismo commit `aced080c5aa038d4e2861398a5da6ac05dd93a74`. En la demostración, la primera fase aprobó seis casos; la segunda detectó exclusivamente R1; la tercera aprobó seis casos después de restaurar el controlador. El job demo concluye aprobado porque verifica esa secuencia, conservando el código 1 de PHPUnit y la assertion fallida en la fase intermedia. Los reportes descargados se publicaron en `docs/evidencia-ci`.

8. Análisis y conclusiones

Esfuerzo y tiempo

El trabajo incluyó lectura de requisitos, inspección del backend, fuentes oficiales, seis pruebas, entorno aislado, script, workflow y documentos de entrega. No se cronometró el esfuerzo humano, por lo que no se asignan horas ficticias. La evidencia mide los comandos de pruebas; no se presenta el tiempo del build como medición porque no se registró por separado.

Las tres fases sumaron 6.612 segundos en esta máquina; cada suite interna tomó menos de medio segundo. La primera fase tuvo mayor duración total por iniciar y esperar PostgreSQL. Ese costo es pequeño para seis casos, pero no se extrapola a toda la suite, al primer build sin caché o al runner remoto. La rapidez observada permite repetir la regresión sin una espera extensa.

Problemas encontrados y solución

El esquema se administra con SQL: se eligió PostgreSQL desde `init.sql` y transacciones para aislamiento. La contradicción entre XML original/SQLite y comentarios de CI se evitó con un XML dedicado que fuerza PostgreSQL. El bloqueo inicial de Docker se resolvió al ampliar permisos del entorno. La advertencia PSR-4 se resolvió alineando la carpeta `Tests/Tarea5` con el namespace; `cacheResult=false` evita escribir en el montaje de pruebas de solo lectura.

Utilidad y alcance

La detección experimental demostró que R1 rechaza la fecha incorrecta sin hacer fallar los otros cinco casos y que restaurar la regla recupera una ejecución aprobada. I1–I3 comprobaron persistencia y coherencia de respuestas. R2 y R3 comprobaron el comportamiento actual de idempotencia y aislamiento; no se mutaron deliberadamente esos dos casos, de modo que su sensibilidad no se demostró mediante experimentos independientes.

La estrategia puede detectar errores de asignación de IDs, pérdida de persistencia, cambios de fechas, incoherencia de saldo/historial, reintentos duplicados y acceso cruzado a notificaciones. Quedan fuera renderizado/navegación Expo, dispositivos móviles, tokens reales de Sanctum, CORS/transporte HTTP, servicios externos, estrés/carga, concurrencia real entre conexiones, auditoría completa de seguridad y rollback de compras fallidas. Las transacciones de los fixtures no demuestran por sí mismas atomicidad frente a fallos.

Conclusión sustentada en resultados

Se mantendría PHPUnit con GitHub Actions: PHPUnit permitió aprovechar el backend existente, ejecutar rápidamente assertions de DB y demostrar una regresión con un oráculo específico. Docker aisló los datos y reprodujo el esquema efectivo. El pipeline conserva reportes incluso en fallos, y su funcionamiento remoto se confirmó con ejecuciones reales en GitHub Actions.

La combinación aporta una base mantenible para los módulos cubiertos. Cuando el proyecto necesite proteger flujos visibles de Expo Web, Playwright puede ampliar la cobertura. Pest sería una alternativa de estilo para PHP, sin asumir que cambiar de sintaxis aporta pruebas adicionales. La aprobación local es evidencia de esta versión y de estos casos, no una garantía de ausencia de defectos en todo el sistema.

9. Fuentes y control de entrega

Fuentes primarias consultadas el 6 de octubre de 2026. Las comparaciones se sintetizaron para las necesidades de ServiGT; no se transcribieron artículos completos.

[1] PHPUnit 12.5 — configuración XML y logging

<https://docs.phpunit.de/en/12.5/xml-configuration-file.html>

[2] PHPUnit 12.5 — opciones de runner y reportes

<https://docs.phpunit.de/en/12.5/textui.html>

[3] Pest — instalación

<https://pestphp.com/docs/installation>

[4] Pest — plugins y soporte Laravel

<https://pestphp.com/docs/plugins>

[5] Playwright — Trace Viewer

<https://playwright.dev/docs/trace-viewer>

[6] GitHub Actions — workflow artifacts

<https://docs.github.com/en/actions/concepts/workflows-and-actions/workflow-artifacts>

[7] GitLab CI/CD — unit test reports

https://docs.gitlab.com/ci/testing/unit_test_reports/

[8] GitLab CI/CD — job artifacts

https://docs.gitlab.com/ci/jobs/job_artifacts/

[9] Jenkins — Pipeline y Docker

<https://www.jenkins.io/doc/book/pipeline/docker/>

[10] Jenkins — JUnit plugin

<https://www.jenkins.io/doc/pipeline/steps/junit/>

[11] GitHub Actions — ejecución manual y rama predeterminada

<https://docs.github.com/en/actions/how-tos/manage-workflow-runs/manually-run-a-workflow>

Fuentes locales: tarea5.pdf, AGENTS.md del proyecto, composer.json/lock, Dockerfile, init.sql, controladores Pedido/Credito/Notificacion, modelos, factories y pruebas Feature relacionadas. La copia no incluye modificaciones humanas sin commit ni archivos privados.

Entrega pendiente: completar integrantes y carnés. El código, el PDF y el video sin voz se publicaron en GitHub; la ejecución de CI se verificó y su evidencia se incorporó a los entregables.